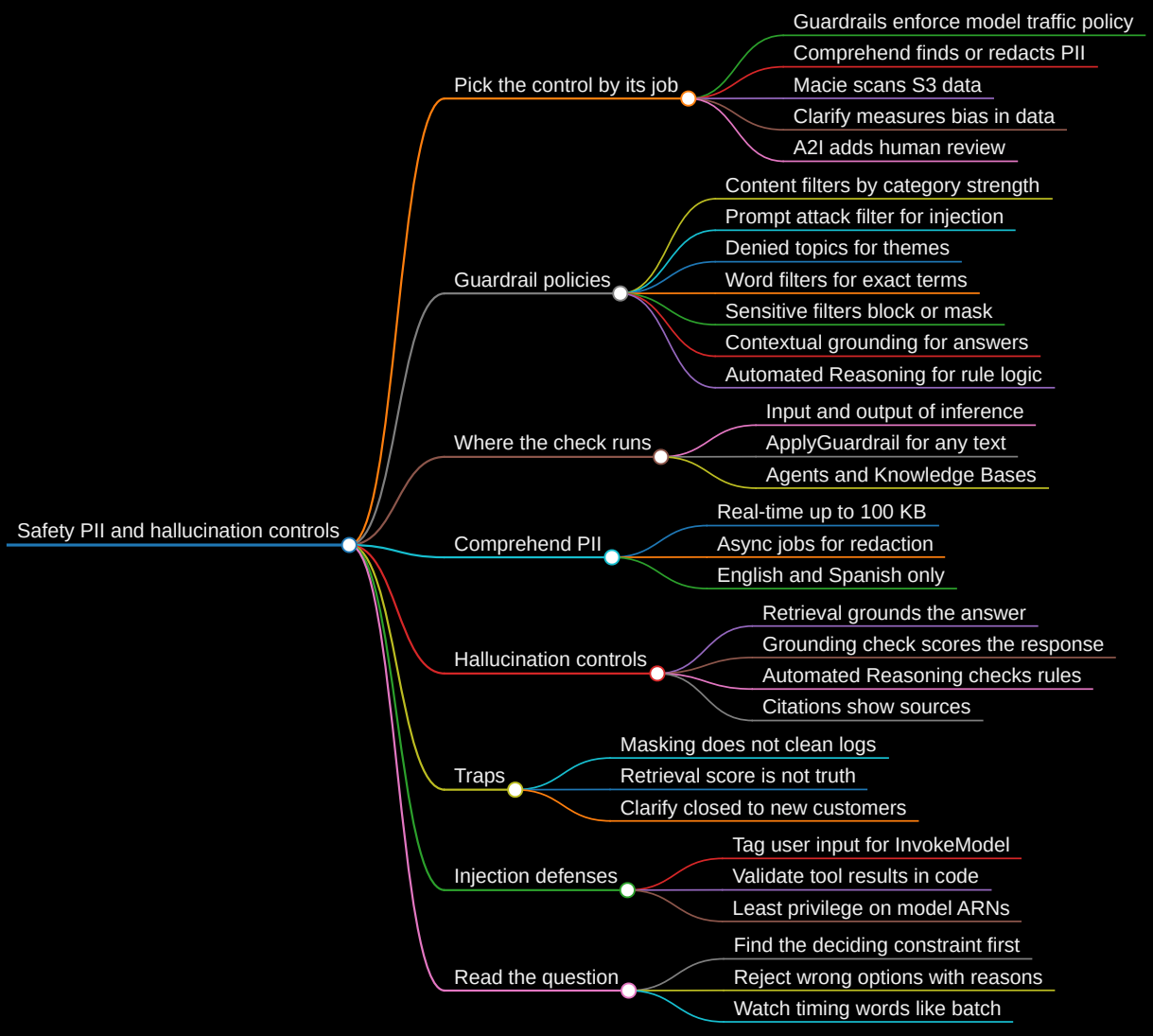
Module m2: Safety, PII, and hallucination controls

After this module, you can read a long scenario, find the constraint that decides the service, and reject each wrong option with one reason. Most confusion comes from three names that sound alike: Amazon Bedrock Guardrails, Amazon Comprehend, and the many "filters" inside an application. This module puts that comparison at the center. Each section tells you when to choose a control, when not to choose it, and which limit decides the answer.



The deciding constraint

Start from the scenario, not from the list of services. Ask five questions in order. Where does the text live, at rest in storage or in a live request? What must happen to it, locate, mask, block, or score? Is a model call part of the path? Does a source or a rule decide whether the answer is correct? Does a person need to decide? The first two answers usually remove most of the options.

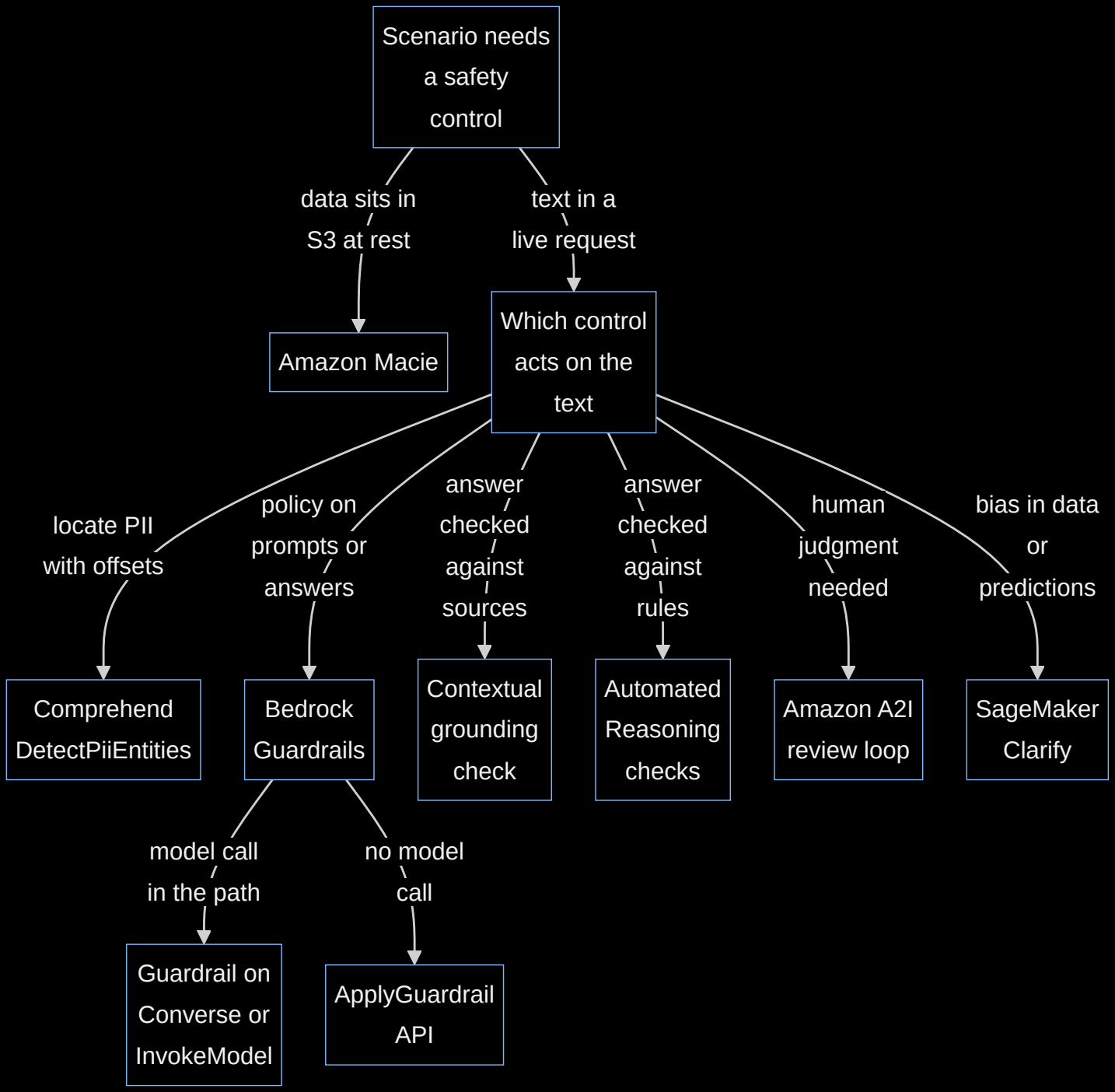


Figure: Start from where the text lives, then from what must happen to it.

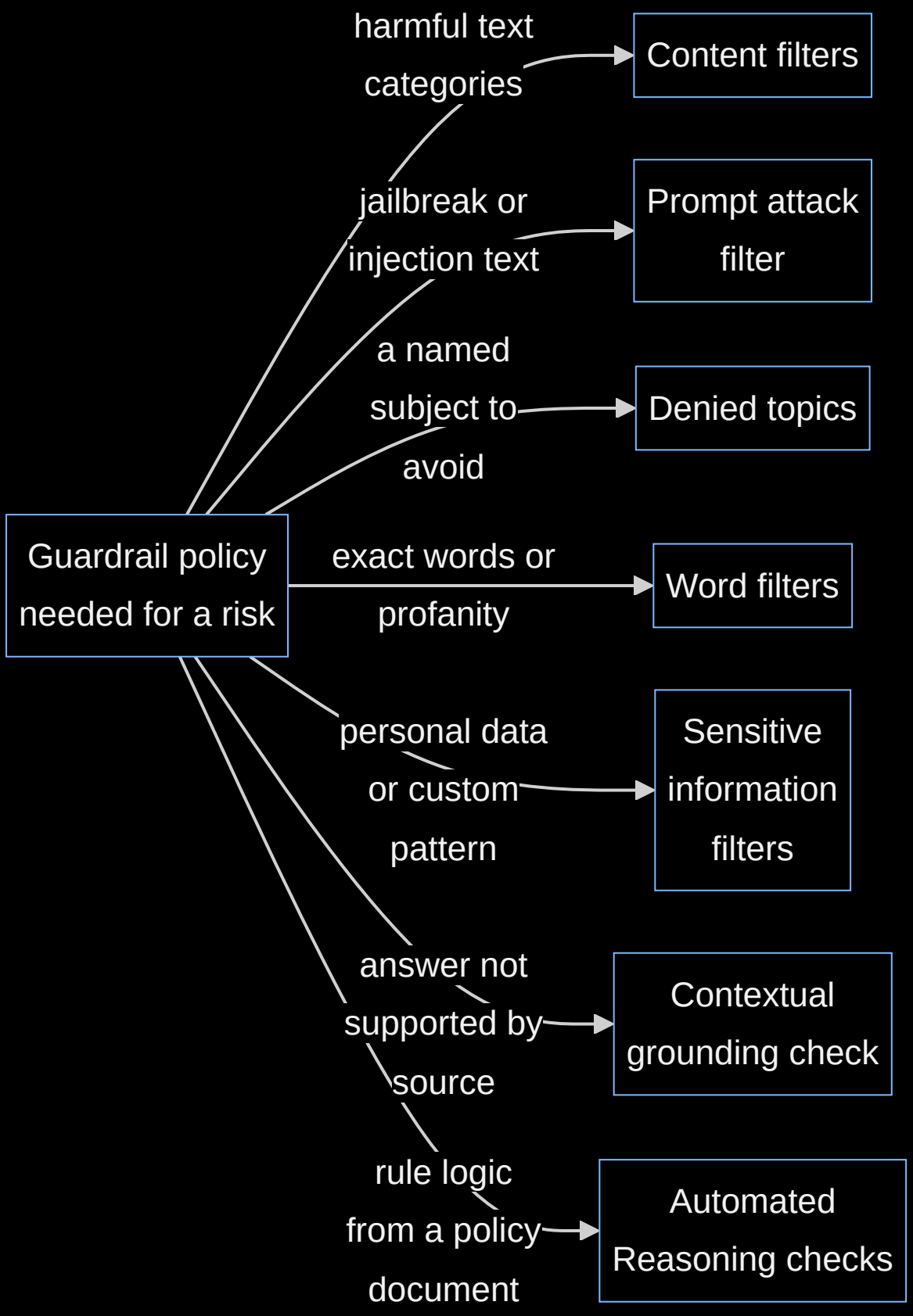


Figure: Each guardrail policy answers one kind of risk.

Signal words for each branch of the tree:

- Signal words for Macie: at rest, S3 bucket, discovery, inventory, object. Macie works on stored objects, not on a live chat message.

- Signal words for Comprehend: PII, redact, locate, offsets, English or Spanish, text. Comprehend returns entities and positions, and your code applies the change.
- Signal words for guardrails: harmful, unsafe, insults, jailbreak, block, mask, denied topic, blocked message. A guardrail returns a decision on both the input and the output.
- Signal words for grounding: source passage, hallucination, supported by the source, relevant to the question. The check needs a grounding source, a query, and the answer.
- Signal words for rules: policy document, eligibility rules, must follow the policy, formal logic. Automated Reasoning returns findings that your application must act on.
- Signal words for humans: reviewer, escalate, human in the loop, approval queue. A2I adds people to the workflow, so it is slower than an automatic block.
- Signal words for bias: demographic group, fairness, imbalance, bias metric. Clarify measures the data or the predictions, not the text of one message.

Amazon Bedrock Guardrails

What it is

Amazon Bedrock Guardrails is a managed set of safeguards that you configure once and apply to model inputs and model outputs. A guardrail holds one or more policies and a blocked message. The policies are content filters, prompt attack detection, word filters, denied topics, Automated Reasoning checks, contextual grounding checks, and sensitive information filters. A guardrail must contain at least one policy and a messaging setting for blocked prompts and blocked responses.

Each policy has its own action. Some actions block content and return the blocked message. Other actions mask content or only report a detection with no change. The console calls the no-change setting Detect. The API calls it NONE.

When to choose it

Rule: Choose Guardrails when the same safety policy must cover model prompts, model answers, or both, and the team wants managed policies instead of custom classifiers.

Guardrails fits a chatbot that needs harmful content blocked, certain topics refused, and personal data masked. The application sets the guardrail identifier and version on the model call, so the same policy applies even when the team changes the prompt text. This is the answer when the wording says "consistent across models" or "minimal custom moderation code."

When not to choose it

Trap: Guardrails does not replace data cleaning before ingestion. A guardrail evaluates a request or response at inference time. It is not a batch redaction tool for a folder of files in S3.

Trap: Guardrails does not cover the reasoning content blocks in the protection described for the content filters. A test answer that depends on protecting reasoning output with the content filters is wrong for that reason.

What it pairs with

Guardrails pairs with Comprehend when the design needs locating or redaction with exact offsets. It pairs with Lambda when a deterministic notice or tool result must be checked. It pairs with CloudWatch and invocation logs when auditors need evidence. It attaches to Agents and Knowledge Bases as well.

Limits and numbers

- Filter strengths for content filters are NONE, LOW, MEDIUM, and HIGH. Each category has an input strength and an output strength.
- Content filters support text and image input for the harmful categories in the console. The API input modalities field accepts TEXT and IMAGE.
- Standard tier extends detection to code. The docs name comments, variable and function names, and string literals.
- Blocked content appears in plain text in invocation logs when logging is on. Plan retention for those logs.

- Contextual grounding threshold is between 0 and 0.99. A threshold of 1 is not valid.
- Contextual grounding limits are 100,000 characters for the source, 1,000 for the query, and 5,000 for the response.
- Not in source: the number of policies or words allowed in one guardrail, and the request rate limits.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-components.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html. Sundog slide page 54 lists the same policy families.

Input versus output evaluation

A guardrail used during inference checks the user input before the model runs. It then checks the model response before the application returns it. If the input is blocked, the model does not produce an answer and the blocked message returns. If the output is blocked or masked, the response that reaches the user changes.

Rule: When the question says both prompts and responses must be checked, configure input and output strength or action separately, and attach the guardrail to the inference call.

Trap: A single strength value is not enough when the scenario allows more false positives on user text but needs strict answers. Set the input and output strengths to different values.

Exam tip: Look for the words "before the model" or "before the response is shown." Those words point to input and output checks, and they also point to a path that must cover both sides.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-components.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters-overview.html

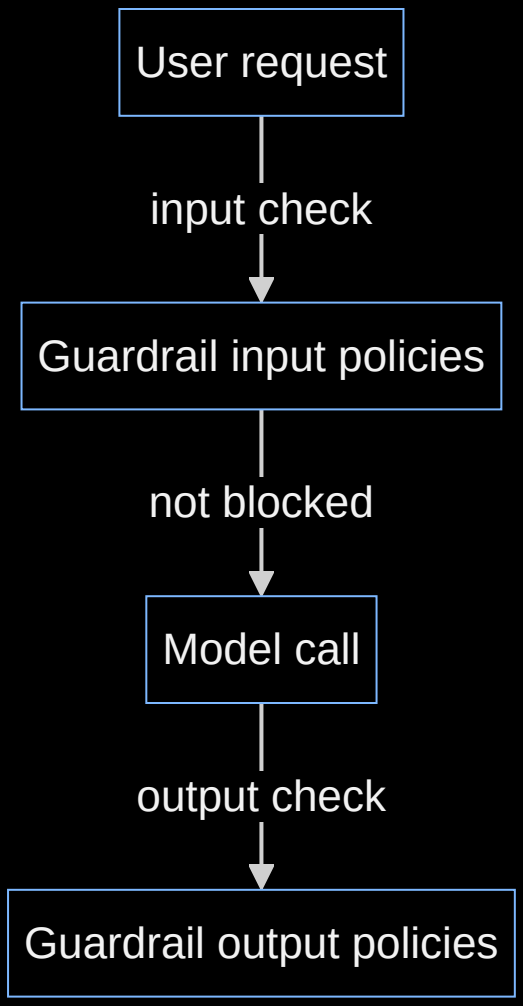


Figure: One guardrail covers the input and the output of one inference call.

Content filters

What it is

Content filters detect harmful text or image content in prompts and responses. The predefined categories are Hate, Insults, Sexual, Violence, and Misconduct. Prompt attack is offered as a category inside content filters. Each category has its own strength for prompts and for responses.

When to choose it

Rule: Choose content filters when the risk is harmful language in a known category and the team wants a managed strength setting, not a list of words.

Use content filters for abusive language from customers, sexual content in a public assistant, or violent text in a generated summary. The strength setting is the control that changes how many borderline messages are blocked. The team does not need to write a classifier.

When not to choose it

Trap: Content filters do not find a named phrase that the team wants to block. Use word filters for an exact phrase, such as a brand name or a banned term.

Trap: Content filters do not evaluate tool results, tool definitions, or model-generated tool call arguments in tool use workloads. The docs say these fields are not assessed.

What it pairs with

Content filters pairs with word filters for banned terms and with denied topics for themes. It is the first policy to check when the scenario lists harassment, hate, or threats. It also pairs with a second control for tool outputs, because the filter does not read those fields.

Limits and numbers

- Strength NONE applies no filtering.
- Strength LOW blocks content with HIGH confidence only.
- Strength MEDIUM blocks content with HIGH and MEDIUM confidence.
- Strength HIGH blocks content with HIGH, MEDIUM, and LOW confidence. This is the strictest setting.
- Standard tier is needed for the code domain. Classic tier covers natural language.
- Not in source: the exact list of languages for each content filter category in the pages reviewed.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters-overview.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters.html

Prompt attack filter

What it is

The prompt attack filter detects jailbreaks, prompt injection, and prompt leakage. A jailbreak tries to bypass the model safety behavior. Prompt injection tries to override the developer instructions. Prompt leakage tries to reveal the

system prompt or developer configuration. Prompt leakage detection needs the Standard tier.

When to choose it

Rule: Choose the prompt attack filter when untrusted user text can try to override instructions, and the team wants a managed classifier on that text.

A typical case is a free-text field that a customer fills in and sends to the model with trusted instructions around it. The filter only needs to evaluate the untrusted part. Scope it correctly with tags or content blocks, as described below.

When not to choose it

Trap: Prompt instructions such as "never reveal your rules" are not enforcement. A model can still follow the injected text. The filter is the managed control.

Trap: The prompt attack filter does not evaluate tool results. A tool response that contains injected instructions needs a control in the application, as described in the injection section.

What it pairs with

It pairs with input tags for InvokeModel and content blocks for Converse. It pairs with application-side validation of tool outputs. It also pairs with the Standard tier when the scenario names prompt leakage.

How to scope it

1. Wrap only the untrusted user text in the input tag for InvokeModel.
2. Set the same tag suffix in the guardrail configuration of that request.
3. Send the user text as a guard content block for Converse.
4. Keep trusted instructions and retrieved passages outside the tags.

The docs say that without input tags, Bedrock does not apply the prompt attack filter on InvokeModel and InvokeModelWithResponseStream. The tag names use the amazon-bedrock-guardrails-guardContent prefix with a suffix.

Trap: Tagging the whole prompt is a scoping mistake. The filter then checks trusted instructions too. Tag only the field that a user controls.

Trap: Raising the misconduct or violence strength does not fix a jailbreak in an untagged InvokeModel body. The prompt attack category needs its own strength and the tags.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-prompt-attack.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html (Converse guard content blocks). Sundog slide page 69 describes prompt injection and prompt leakage in general terms.

Denied topics

What it is

A denied topic is a named subject that the application must avoid, described in natural language. An example is advice on a regulated subject. The guardrail uses the topic definition and examples to decide whether a prompt or response falls inside that topic.

When to choose it

Rule: Choose denied topics when the rule is a subject area, such as investment advice, and not a fixed word or a fixed pattern.

A bank assistant that must refuse to give investment guidance is a clean case. The team describes the topic once in the guardrail. Every model behind the guardrail then refuses the same subject.

When not to choose it

Trap: Denied topics are a poor fit for an exact list of words or for entity names. Word filters or sensitive information filters are the right controls for those rules.

Trap: A denied topic does not measure whether an answer is true. Use contextual grounding or Automated Reasoning for correctness.

What it pairs with

Denied topics pairs with content filters for harmful categories and with word filters for exact terms. It also pairs with the blocked message, which the user

sees when the topic is refused.

Limits and numbers

- Denied topics use a natural language definition and optional examples.
- Not in source: the maximum number of denied topics per guardrail in the pages reviewed.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-components.html. Sundog slide page 281 describes blocked topics for an enterprise assistant.

Word filters and profanity

What it is

Word filters block exact words or phrases. The team can add custom words, such as a competitor name, or use the managed profanity list. The matching is exact, so the filter does not understand meaning.

When to choose it

Rule: Choose word filters when the banned item is a list of exact terms or the managed profanity list.

A retailer that must block a set of banned slang words is a clean case. A word filter is simple, cheap to run, and easy to audit.

When not to choose it

Trap: Word filters do not catch a rephrased insult. A rude message written with different words passes an exact-match list. Content filters catch the meaning of harmful language.

Trap: Word filters do not find personal data formats. A phone number or a national ID needs sensitive information filters or a custom regex.

What it pairs with

Word filters pairs with content filters because the two catch different things. The exact list catches known terms, and the managed category catches new phrasing.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-components.html (word filters described as exact match, with a ready-to-use profanity option).

Sensitive information filters

What it is

Sensitive information filters detect personal data in prompts and responses. The filter is a probabilistic machine learning feature that uses context. It supports a set of predefined PII types and custom regex patterns. For each type, the action is BLOCK, ANONYMIZE, or NONE in the API. The console labels the masking action Mask.

When to choose it

Rule: Choose sensitive information filters when the policy names PII types or a custom identifier pattern, and the control must sit on model input or output.

A support assistant that must hide names, emails, and phone numbers in its answers is a clean case. A custom regex covers a loyalty number or a booking ID that has a fixed format.

When not to choose it

Trap: Masking in a guardrail does not sanitize invocation logs. The docs say the input field of the logs keeps the original request. Clean the data before the call or protect the logs with a separate control.

Trap: The filter evaluates text only. It does not evaluate PII that the model writes into tool call arguments, tool results from the application, or tool definitions.

Trap: The guardrail trace keeps the unmasked PII value in its match field. Treat the trace as sensitive.

What it pairs with

It pairs with Comprehend when the application must replace exact spans before the model call. It pairs with invocation logging when auditors need the record. It pairs with a custom regex when the identifier is company-specific.

Limits and numbers

- Custom regex pattern length is 1 to 500 characters.
- Custom regex does not support lookaround matches.
- Masked text uses the PII type as a placeholder, such as NAME or EMAIL.
- Not in source: the exact count of custom regex patterns per guardrail.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html, https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html. Sundog slide page 54 lists PII removal and masking as a guardrail feature.

Contextual grounding check

What it is

The contextual grounding check scores a model response on two measures. Grounding asks whether the response is supported by the source you supply. Relevance asks whether the response answers the query. Each measure has a threshold. A response with a score below the threshold is filtered.

The check needs three inputs. The grounding source is the passage. The query is the user question. The content to guard is the model response. The check runs on the output only, not on the prompt.

When to choose it

Rule: Choose the contextual grounding check when the answer must be supported by a supplied passage, and the team wants a managed score and a block or flag decision.

A retrieval-based assistant that answers from policy pages is a clean case. The application passes the retrieved passages as the source, the question from the user as the query, and the answer as the content to guard.

When not to choose it

Trap: The check needs a source. A general question with no passage cannot be scored for grounding.

Trap: Classic tier is strict. It lowers the grounding score for reasonable inference and conversational phrasing. Classic tier does not support conversational use cases. Standard tier supports reasoning and conversational content but needs cross-Region inference.

Trap: A threshold of 1 is invalid. Use a value from 0 to 0.99.

What it pairs with

It pairs with RAG retrieval, which supplies the source. It pairs with Automated Reasoning when the answer must follow formal rules. It pairs with citations, which show where the passages came from. It does not replace reviewing the answer when the stake is high.

Limits and numbers

- Threshold range is 0 to 0.99.
- Maximum grounding source is 100,000 characters.
- Maximum query is 1,000 characters.
- Maximum response is 5,000 characters.
- Standard tier requires cross-Region inference.
- Not in source: a recommended threshold for Classic tier. The docs suggest 0.7 as a starting point only for Standard tier conversational use.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html. Sundog slide page 54 describes grounding and relevance in the same terms.

Automated Reasoning checks

What it is

Automated Reasoning checks validate model content against formal rules that you derive from a policy document. The policy document is natural language,

such as a rules page. Bedrock extracts rules and variables from it. At runtime, the check returns findings that say whether the content is valid, invalid, or outside the policy scope. The findings include feedback that the application can use to rewrite or escalate.

When to choose it

Rule: Choose Automated Reasoning checks when the answer must follow written rules with conditions, such as eligibility or a benefit formula, and an audit needs a verifiable explanation.

A benefits assistant that must apply a written eligibility rule is a clean case. The check can show which rule and which variable supported the result.

When not to choose it

Trap: The check is detect mode only. It does not block content by itself. The application must read the findings and decide what to do.

Trap: The check does not support streaming. It needs the complete response.

Trap: The check does not detect prompt injection or off-topic content. Use the prompt attack filter and denied topics for those risks.

Trap: A valid result covers only the statements captured by policy variables. Other statements are outside the check.

What it pairs with

It pairs with content filters and topic policies in the same guardrail. It pairs with an application step that uses the findings. It pairs with a test set that checks the policy before release.

Limits and numbers

- Input policy document limit is 5 MB and 50,000 characters.
- The check supports English US only.
- The check is charged per validation request, whatever the result.
- The check is listed as generally available in seven Regions, including US East N. Virginia and EU Ireland.

- Not in source: the exact latency added per request.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html. Sundog slide page 55 describes the policy document flow and the Create policy API.

ApplyGuardrail API

What it is

ApplyGuardrail evaluates any text against a configured guardrail without invoking a foundation model. The request names the guardrail identifier and version, sets the source to INPUT or OUTPUT, and sends the content. The response reports an action of GUARDRAIL_INTERVENED or NONE. The outputs are masked text, the blocked message, or an empty list when nothing happened.

When to choose it

Rule: Choose ApplyGuardrail when text must be checked with the same policy but no model call is part of the path.

Examples are a deterministic notice assembled by Lambda, a rules-engine draft, text from an upstream workflow step, or a RAG query checked before retrieval. The policy stays the same as the one used on inference.

When not to choose it

Trap: ApplyGuardrail does not stop a model call by itself. The application must read the action and skip the call.

Trap: An empty outputs list does not mean the content was blocked. It means no action happened.

Trap: Using a model to rewrite deterministic text for safety adds cost and changes the message. Use ApplyGuardrail instead.

What it pairs with

It pairs with Lambda and Step Functions for workflow steps. It pairs with a guardrail on inference for the model path. It pairs with the full output scope

when the team needs to see non-detected entries for debugging.

Limits and numbers

- Source values are INPUT and OUTPUT.
- The output scope FULL returns non-detected entries as well as detected ones. The docs say FULL does not apply to word filters or regex in sensitive information filters.
- The response usage fields count units for each policy type.
- Not in source: image input limits for ApplyGuardrail in the pages reviewed.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-use-independent-api.html. Sundog slide page 21 lists guardrail configuration on Converse.

Guardrail versions

What it is

A guardrail has an identifier and a version. The draft version is used while the team edits the guardrail. The application references a version on each model call or ApplyGuardrail call. A numbered version is a fixed point that a release can point to.

When to choose it

Rule: Pin the application to a numbered version for a release, and test the draft version separately.

This keeps the audit trail clear. The team can prove which policy set was live on a given date.

When not to choose it

Trap: Editing the draft changes what the draft version does. A release that uses the draft does not have a fixed policy set.

Trap: The version must be included with the identifier. An identifier alone is not enough for the call.

Limits and numbers

- The docs show DRAFT as a valid version value in ApplyGuardrail examples.
- Not in source: the rules for creating numbered versions, and any claim that numbered versions are immutable, in the pages reviewed.
- Not in source: how organization-level policies reference guardrail versions. Do not answer a test item on that basis without the policy document.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-use-independent-api.html (version in the request path). Sundog slide page 21 lists the guardrail identifier and version fields on Converse.

Guardrails with Agents and Knowledge Bases

What it is

The Sundog course slides state that a guardrail can be incorporated into agents and knowledge bases. The course example combines an agent, a knowledge base for source material, an action group for a tool, and guardrails. Agent traces include a guardrail step that shows what the guardrail did.

When to choose it

Rule: Attach the guardrail at the agent or the knowledge base layer when the requirement covers every model call in that workflow.

This avoids forgetting the guardrail on one call path.

When not to choose it

Trap: The pages reviewed do not show a guardrail that scans retrieved chunks before generation. Do not assume that retrieved references are inspected for policy. Check the retrieved passages with ApplyGuardrail or with a grounding check if the scenario needs it.

Trap: A citation shows where a passage came from. It does not prove that the answer is correct.

Limits and numbers

- Agent traces include a guardrail trace type.

- Not in source: the exact knowledge base retrieval fields that show guardrail results.

Source: Sundog slide pages 54, 226, and 419. The AWS documentation for agent guardrail configuration was not part of the fetched set.

Model invocation logging as audit evidence

What it is

Model invocation logging records the request and response data for supported Bedrock calls in one Region. The supported operations are Converse, ConverseStream, InvokeModel, and InvokeModelWithResponseStream. The destinations are CloudWatch Logs and Amazon S3, and each destination must sit in the account and Region of the call. Logging is off by default.

Each record includes the model identifier, the caller identity ARN, the token counts, and an optional request metadata object. The caller supplies the request metadata. The input body keeps the original request.

When to choose it

Rule: Enable invocation logging when auditors need prompt and response evidence, and make sure the stored content is already safe to keep.

Use request metadata for a case identifier. Use the caller identity for who invoked the model.

When not to choose it

Trap: CloudTrail keeps an API activity record: the caller and the time. It does not replace the prompt and response record that invocation logging keeps.

Trap: The docs say invocation logging is only supported for the bedrock-runtime endpoint. Calls through another endpoint are not captured.

Limits and numbers

- Inline bodies up to 100 KB appear in the record. Larger bodies and binary data go to S3 as separate objects.

- Logging is disabled by default.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html. Sundog slide page 446 lists Bedrock model invocation logs as a monitoring source.

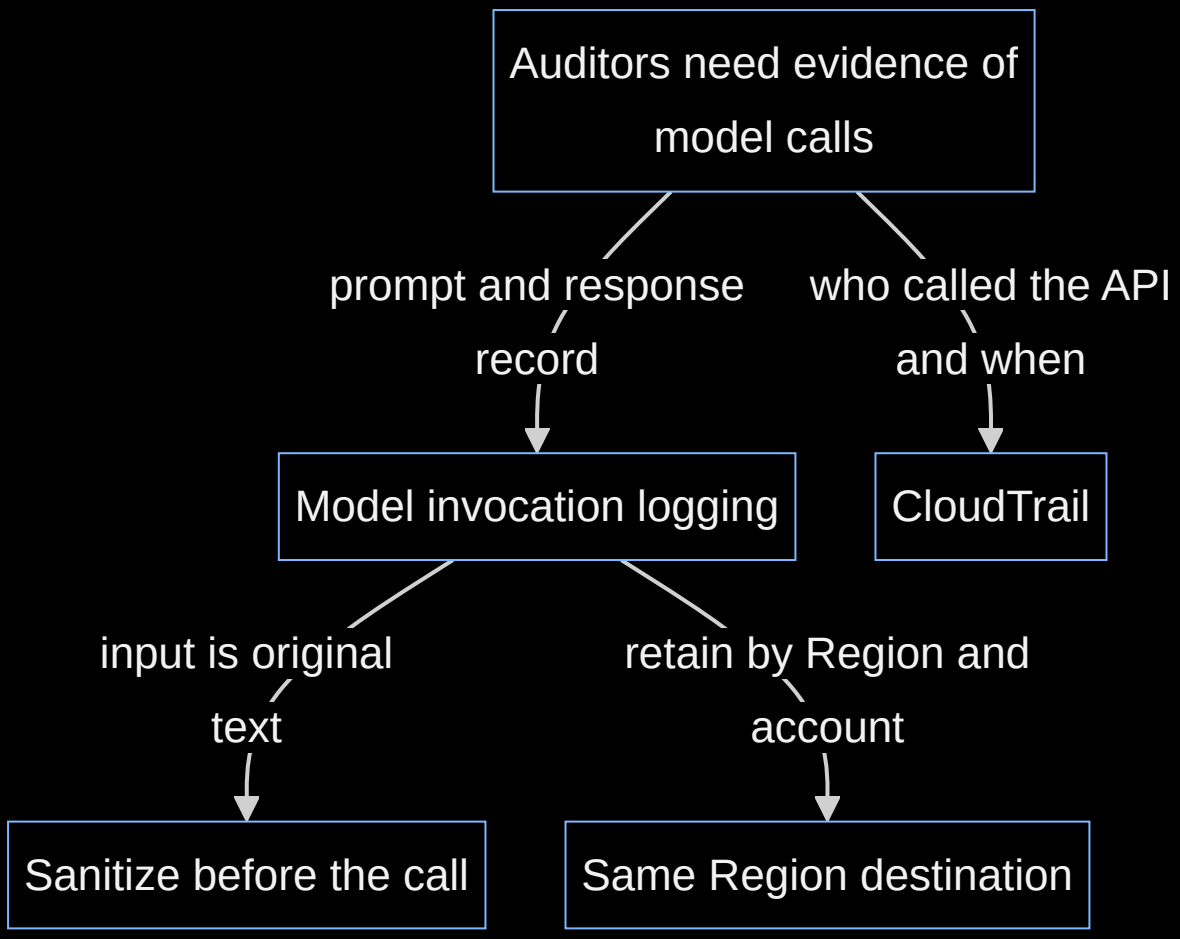


Figure: Logging keeps evidence, so the content must already be safe to keep.

Amazon Comprehend PII detection and redaction

What it is

Amazon Comprehend detects PII entities in English and Spanish text. It returns each entity with a type, a confidence score, and character offsets for the start and end. Real-time analysis handles a single document of up to 100 KB of UTF-8 text. Redaction returns a copy of the text with the entities masked, and the docs say this needs an asynchronous batch job.

When to choose it

Rule: Choose Comprehend when the application must find PII in text before the model sees it, and the application must control the replacement.

Lambda can call DetectPiiEntities, use the offsets to replace each span with a placeholder, and then call the model. The stored audit record can keep the type, score, and offsets for each entity.

When not to choose it

Trap: A real-time PII call returns locations, not redacted text. Writing the response straight into the prompt sends the original text.

Trap: The asynchronous redaction job is a batch step. It does not fit a live chat path that must answer in seconds.

Trap: Comprehend does not evaluate tool results in a guardrail sense. Your code must call it on each tool result that enters the model context.

What it pairs with

It pairs with Lambda for pre-processing and post-processing. It pairs with a Bedrock guardrail when the team wants a second layer on the model output. It pairs with Transcribe when the source is audio, because Transcribe can also redact PII from transcripts.

Limits and numbers

- English and Spanish are supported for PII.
- Real-time input is up to 100 KB of UTF-8 text.
- Each detected entity has BeginOffset, EndOffset, Type, and Score.
- Not in source: the redaction mode names and masking options for the asynchronous job in the pages reviewed.
- Not in source: the exact contents of the ContainsPiiEntities response. Do not assume it returns offsets.

Source: https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html, https://docs.aws.amazon.com/comprehend/latest/dg/trust-safety.html. Sundog slide pages 57, 111, and 118 describe Comprehend in Lambda pre-processing and token-level redaction.

Amazon Comprehend toxicity and prompt safety

What it is

DetectToxicContent is a synchronous API for English text. It takes up to 10 text strings, each up to 1 KB. It returns categories such as insult, harassment or abuse, hate speech, violence or threat, profanity, sexual content, and graphic content, with scores and a toxicity score.

Prompt safety classification uses the ClassifyDocument operation on English text up to 10 KB. It returns SAFE or UNSAFE with a score. The docs state that topic modeling, event detection, and prompt safety classification are no longer available to new customers.

When to choose it

Rule: Choose DetectToxicContent for short English chat turns when the team needs a score per turn before the model call and a Comprehend-based check is allowed.

When not to choose it

Trap: Do not choose Comprehend toxicity for Spanish or long documents. The language and size limits fail the requirement.

Trap: Do not design a new solution around prompt safety classification for a new account. The docs say it is not available to new customers.

Trap: A toxicity score is not a guardrail. It does not block the model call by itself. The application must act on the score.

Limits and numbers

- DetectToxicContent: English only, up to 10 strings, each up to 1 KB.
- ClassifyDocument prompt safety: English, up to 10 KB, classes SAFE and UNSAFE.
- Prompt safety and topic modeling are not available to new customers.

Source: https://docs.aws.amazon.com/comprehend/latest/dg/trust-safety.html

Amazon Macie

What it is

Amazon Macie finds and reports sensitive data in Amazon S3. It discovers sensitive data in S3 objects by using managed data identifiers and custom data identifiers. It can run automated sensitive data discovery with sampling, or run sensitive data discovery jobs. Findings go to EventBridge and Security Hub.

When to choose it

Rule: Choose Macie when the data is stored in S3 and the team must find where sensitive data lives before the data is used.

Macie fits the step before a RAG index is built from a folder of support tickets. Findings can route high-risk objects to a quarantine prefix.

When not to choose it

Trap: Macie does not scan a live chat message. The fetched page describes S3 scope only. A live request needs Comprehend, a guardrail, or application code.

Trap: Macie finds data. It does not redact a file for you by itself. Pair it with a redaction step.

Limits and numbers

- Scope is S3 general purpose buckets.
- Findings can be sent to EventBridge.
- Not in source: the per-object size limit for discovery in the pages reviewed.

Source: https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html. Sundog slide pages 499 and 11 describe Macie for S3 discovery and PII.

Amazon SageMaker Clarify bias

What it is

SageMaker Clarify measures bias in data before training and in predictions after training. Pre-training metrics include class imbalance, difference in proportions of labels, and several divergence measures. Clarify can also run bias drift

monitoring against a baseline. The Sundog slides list these metrics and the terms facet, label, and group variable.

Trap: The docs state that Clarify is no longer open to new customers. Existing customers can continue to use it, but the docs say no new features are planned. Do not design a new workload around Clarify if the scenario asks for a new account.

When to choose it

Rule: Choose Clarify when the question asks about bias in a dataset or in predictions across demographic groups, and the account already uses it.

When not to choose it

Trap: Clarify does not check a single generated sentence for harm. A guardrail or a judge model does that.

Trap: Pre-training metrics do not need model outputs. Post-training or drift metrics do need predictions.

Limits and numbers

- Pre-training metrics include CI, DPL, KL, JS, LP, TVD, KS, and CDD.
- Clarify is no longer open to new customers.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-detect-data-bias.html. Sundog slide pages 327, 328, and 330.

Amazon Augmented AI human review

What it is

Amazon Augmented AI (A2I) inserts humans into a workflow to review and correct results. The Sundog slides list it in the set of tools for responsible AI, next to Clarify and Model Monitor. The AWS docs page for A2I was not part of the fetched set, so the flow details below are not confirmed.

When to choose it

Rule: Choose A2I when a person must review a model result before it is final, such as a borderline decision with a regulated outcome.

When not to choose it

Trap: A2I is not a real-time block. A reviewer takes minutes or hours, so it cannot be the only control in a live chat path.

Trap: A guardrail blocks automatically. A2I routes to a person. Pick the one that matches the wording of the requirement.

Limits and numbers

- Not in source: review queue limits, work team sizes, and integration details beyond the slide statement.

Source: Sundog slide page 433. AWS documentation for A2I was not confirmed in this review.

Prompt injection defenses beyond guardrails

What it is

Prompt injection is an application security problem. Bedrock gives a filter, but the application must tag the input, scope the check, and treat outside text as untrusted. The Sundog slides also describe a system prompt with an instruction to refuse certain requests, and a PII filter or a rule not to store PII.

When to choose it

Rule: Add an application control when the untrusted text reaches a place the guardrail does not evaluate, such as tool results or retrieved documents.

What to do

1. Validate each tool result in Lambda before it re-enters the model context.
2. Keep tool permissions to the approved model ARNs and tool actions.
3. Tag only the user-controlled text for InvokeModel prompt attack checks.
4. Log each blocked attempt and send it to a review queue.

When not to choose it

Trap: A longer system prompt is not a control. It can be overridden by the injected text.

Trap: A WAF rule matches strings at the edge. It does not understand a role-play request written in fluent language.

Limits and numbers

- The prompt attack filter does not evaluate tool results or tool definitions.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-prompt-attack.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html. Sundog slide page 69 describes prompt injection, prompt leakage, and the system prompt approach.

Hallucination controls

What it is

Hallucination controls are layers. Retrieval-augmented generation supplies source passages so the model answers from them. A grounding check scores the answer against those passages and the query. Automated Reasoning checks an answer against written rules. Citations show which passages the answer used. Each layer answers a different question.

The Sundog slides say RAG can prevent hallucinations about information the model was not trained on, and can still hallucinate. The slides also say RAG is sensitive to the relevance of the retrieved passages.

When to choose each layer

- RAG: the question is about private or new documents.
- Grounding check: the answer must be supported by the passages, and the team wants a managed score.
- Automated Reasoning: the answer must follow written rules with conditions.
- Citations: the user or reviewer must see the source.

When not to choose each layer

Trap: A retrieval score shows relevance to the query. It is not a truth score. A high score does not prove that the answer is correct.

Trap: A citation proves a link, not a supported claim. Pair it with a grounding check when the claim must be verified.

Trap: Model evaluation jobs measure quality across a test set. They do not moderate one live answer.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html. Sundog slide pages 162, 431 describe RAG pros and cons and output validation. Citation field names: Not in source.

Watermark detection for Titan images

Not in source. The Sundog extract and the pages fetched for this module do not describe watermark detection for Titan images. Do not answer a test item on watermark detection from this module.

Source: none for this topic. Checked the Sundog extract and the module corpus. No match.

Guardrails versus Comprehend

This is the center comparison of the module. Guardrails applies a policy to model traffic and returns a decision. Comprehend analyzes text and returns entities or scores that the application uses. A guardrail can mask PII in a response, but it does not return the offsets that a redaction step needs. Comprehend can find PII, but it does not block a harmful prompt for you.

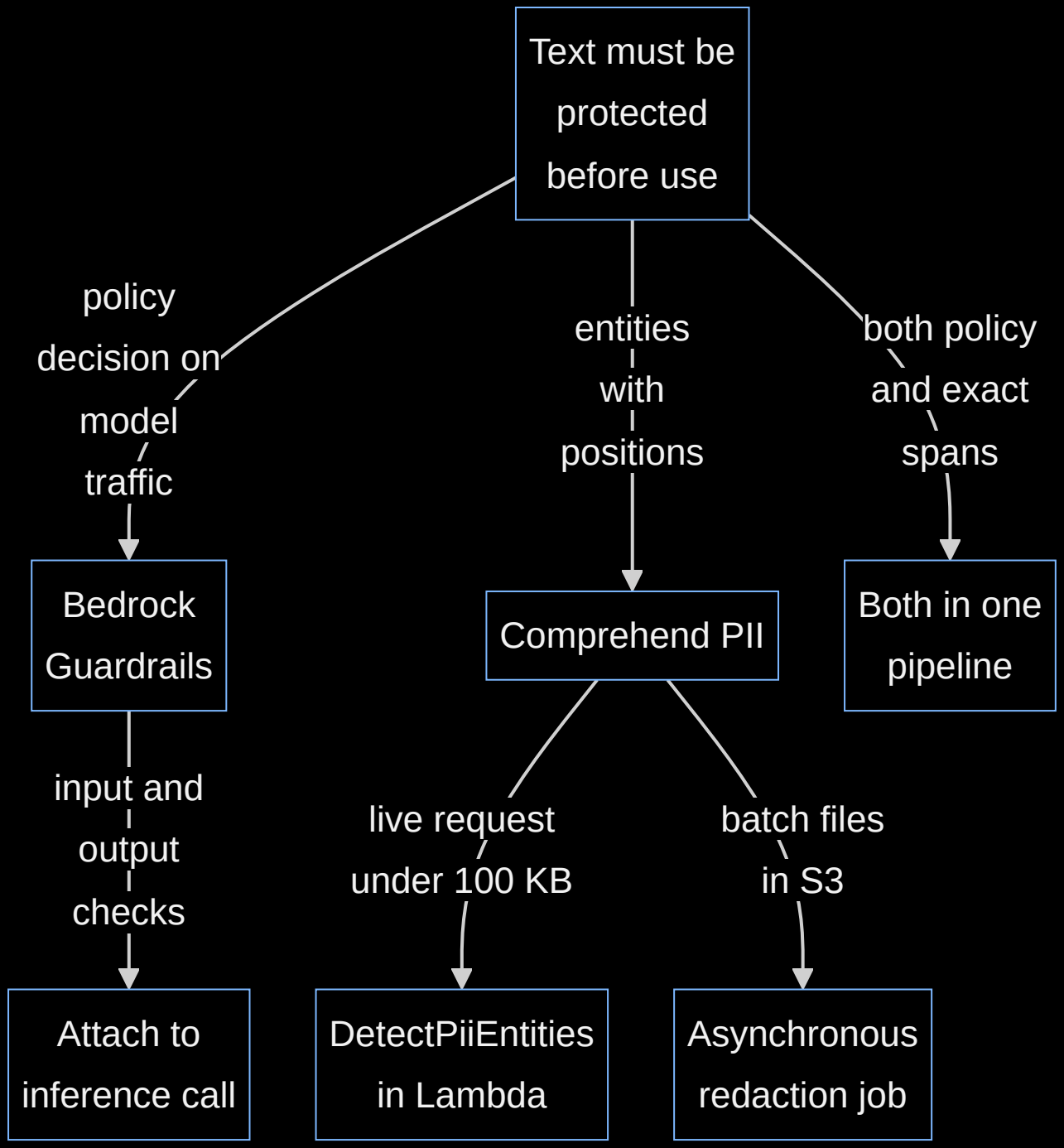


Figure: Guardrails decides, Comprehend locates. Use both when the scenario asks for both.

- Purpose. Guardrails: enforce a policy on prompts and responses. Comprehend: find, score, or redact entities in text.
- Input. Guardrails: the prompt, the response, or any text sent to ApplyGuardrail. Comprehend: one text document or a batch of files.
- Output or effect. Guardrails: allow, block, mask, or report. Comprehend: entity types, scores, offsets, or a redacted copy from a batch job.

Exam tip: Read the verb. "Block" or "enforce" points to Guardrails. "Identify the offsets" or "produce a redacted file" points to Comprehend.

Rule: If the answer must change the text before the model sees it, and the change must be exact, use Comprehend plus application code. If the answer must refuse or mask on policy, use Guardrails.

The one question to ask yourself: Does the requirement name a policy outcome, or a location of data?

Trap: Choosing Comprehend because the scenario says "PII." When the same sentence also says "block harmful content" and "consistent across models," Guardrails is the answer. Comprehend alone is incomplete.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-components.html, https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html. Sundog slide pages 54 and 57.

Content filters versus word filters versus denied topics versus sensitive filters

Learners often treat all four as "filters." They are four different policy types in one guardrail. Each one has its own input, its own rule type, and its own action. Pick the one that matches the kind of risk named in the stem.

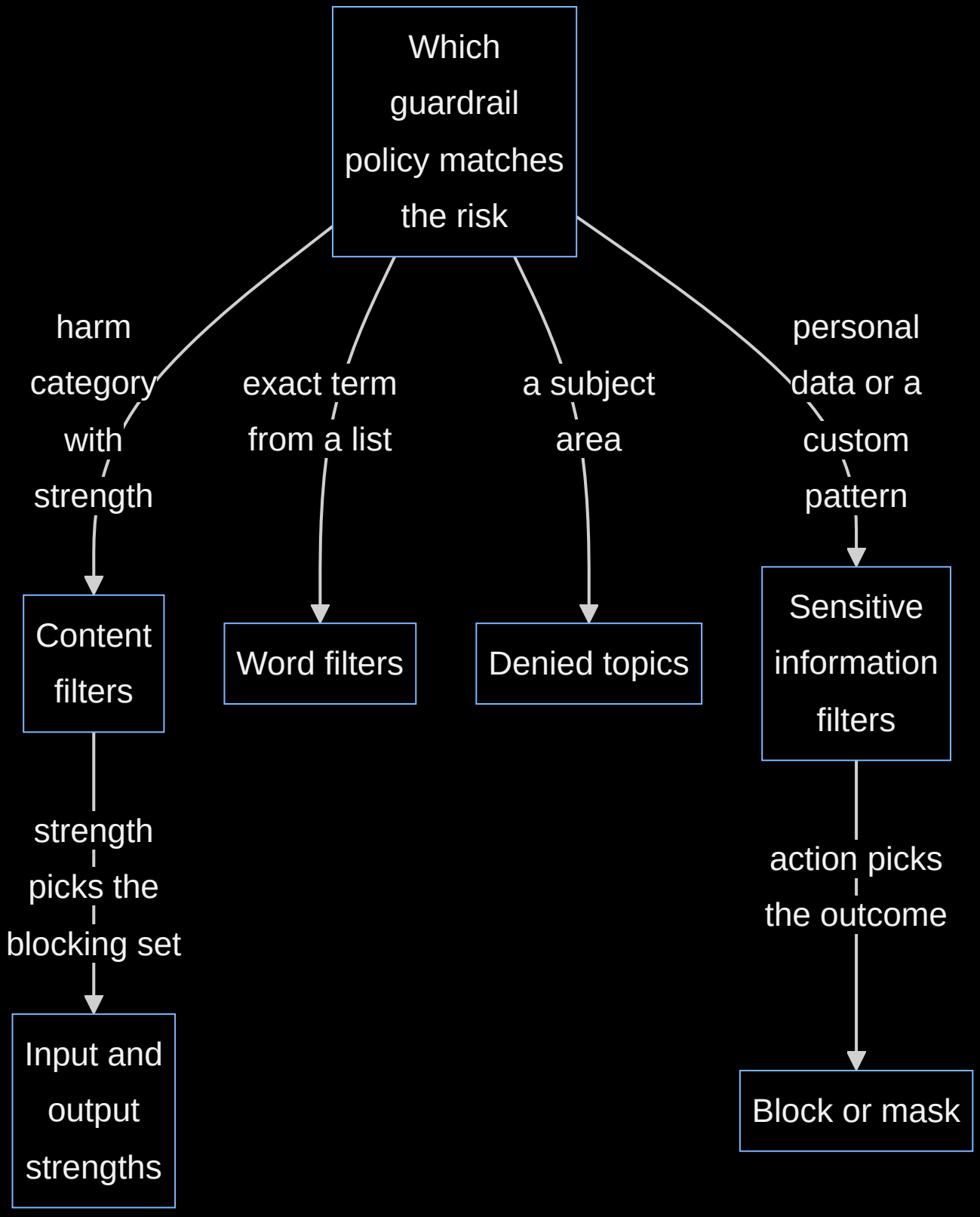


Figure: Match the risk type first, then choose the strength or the action.

- Purpose. Content filters: harmful categories. Word filters: exact terms. Denied topics: subjects. Sensitive filters: personal data and patterns.
- Input. Content filters: text and images by category. Word filters: text by exact match. Denied topics: text described by definition. Sensitive filters: text by PII type or regex.

- Output or effect. Content filters: block by confidence and strength. Word filters: block. Denied topics: block. Sensitive filters: block or mask.

Exam tip: The strength setting belongs only to content filters. If the stem asks for a strength, the answer is content filters or the prompt attack category.

Rule: Use content filters for harm categories, word filters for named terms, denied topics for subjects, and sensitive filters for personal data.

The one question to ask yourself: Is the risk a category, a term, a subject, or a data format?

Trap: A denied topic for a list of banned words. Denied topics are described in natural language. A list of exact words belongs in word filters.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-components.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters-overview.html.

Comprehend real-time versus asynchronous

Comprehend has two timing modes for PII. Real-time analysis returns results in the same call. An asynchronous job reads many documents and writes results later. Redaction is the job that needs the asynchronous mode.

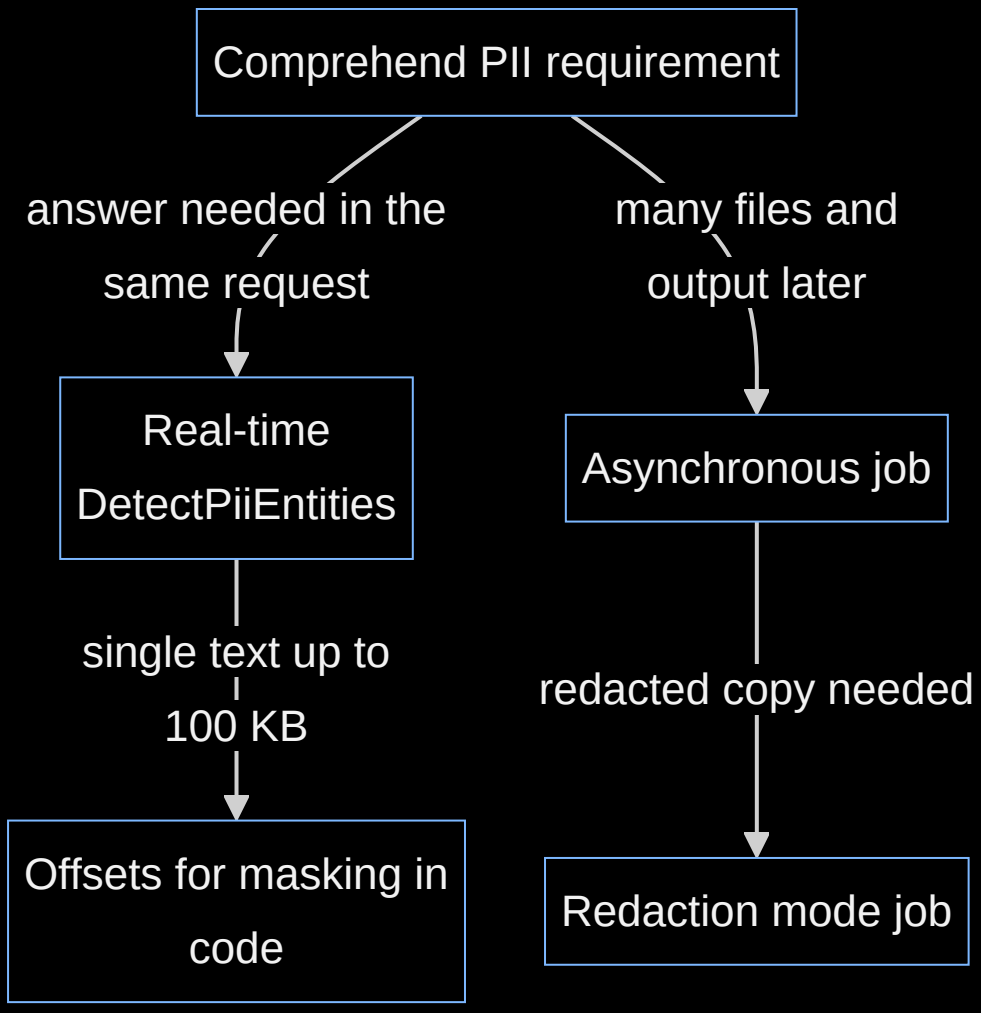


Figure: Timing words in the stem decide the mode.

- Purpose. Real-time: locate entities now. Asynchronous: process a collection later.
- Input. Real-time: one text document up to 100 KB of UTF-8. Asynchronous: a set of documents in S3.
- Output or effect. Real-time: entity list with type, score, and offsets. Asynchronous: output files, including redacted text when the job is a redaction job.

Exam tip: Words such as "hourly," "nightly," or "batch" point to the asynchronous mode. Words such as "chat," "before the model call," or "live" point to real-time.

Rule: Use real-time for a live path, and use the asynchronous job when the files are large or the output can wait.

The one question to ask yourself: Does the answer have to exist before the model call returns?

Trap: Using the real-time response as if it were redacted text. It is a list of locations.

Source: https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html.

Comprehend versus Macie

Both services find sensitive data. Comprehend reads text in a request or a file that you send to it. Macie scans S3 objects where the data already lives, and it reports where sensitive data sits across the bucket inventory.

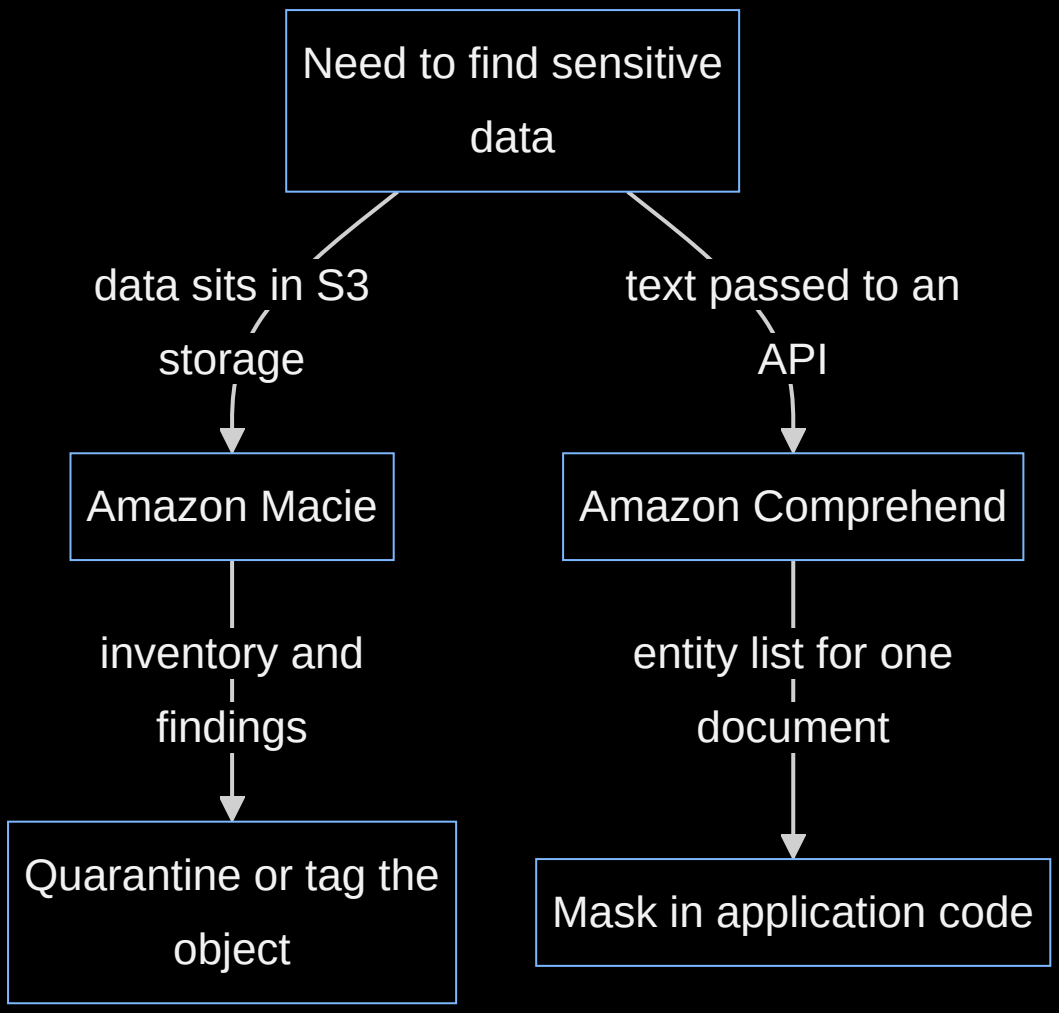


Figure: Storage location decides Macie. Request text decides Comprehend.

- Purpose. Macie: discover and report sensitive data in S3. Comprehend: detect and redact entities in text.
- Input. Macie: S3 objects and buckets. Comprehend: text strings or documents you send.
- Output or effect. Macie: findings and inventory. Comprehend: entity list, scores, offsets, or a redacted copy.

Exam tip: The phrase "at rest" or "already stored" is the Macie signal. The phrase "before the model" or "in the request" is the Comprehend signal.

Rule: Choose Macie to find data in S3 storage. Choose Comprehend to act on text before a model call.

The one question to ask yourself: Is the data sitting in storage, or passing through my code?

Trap: Using Macie to block a live prompt. Macie does not sit in the request path.

Source: https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html, https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html. Sundog slide page 499.

ApplyGuardrail versus a guardrail on inference

Both use the same guardrail. The difference is the call path. A guardrail on inference is attached to a model call, so the input and output checks happen inside that call. ApplyGuardrail checks text on its own, so it works with any text and with no model call.

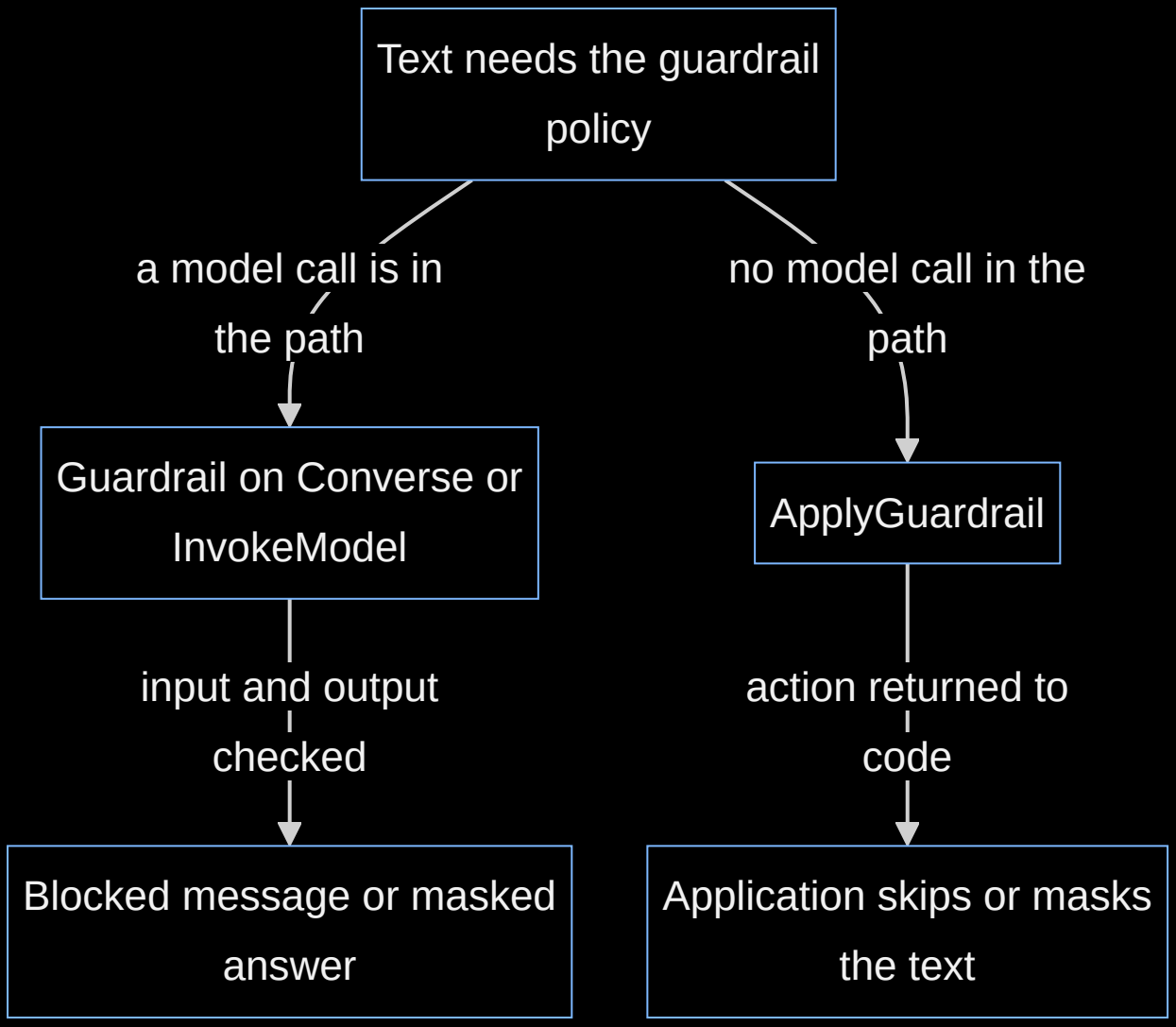


Figure: The model call decides which path to use.

- Purpose. Inference guardrail: protect a model call. ApplyGuardrail: protect any text.
- Input. Inference guardrail: prompt and response. ApplyGuardrail: any text with a source value of INPUT or OUTPUT.
- Output or effect. Inference guardrail: the model answer changes. ApplyGuardrail: an action that your code must use.

Exam tip: The phrase "without invoking a foundation model" points to ApplyGuardrail. The phrase "every model call" points to the inference attachment.

Rule: Use both when the scenario has a model path and a non-model path under one policy.

The one question to ask yourself: Is there a model call for this text?

Trap: Calling ApplyGuardrail after the model returns and assuming the input was checked. Input and output need separate checks in that design.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-use-independent-api.html. Sundog slide page 21 (Converse guardrail configuration).

Contextual grounding versus Automated Reasoning versus citations

These three answer different questions about an answer. Grounding asks whether the answer is supported by the passage and relevant to the query. Automated Reasoning asks whether the answer follows written rules. Citations ask where the answer came from.

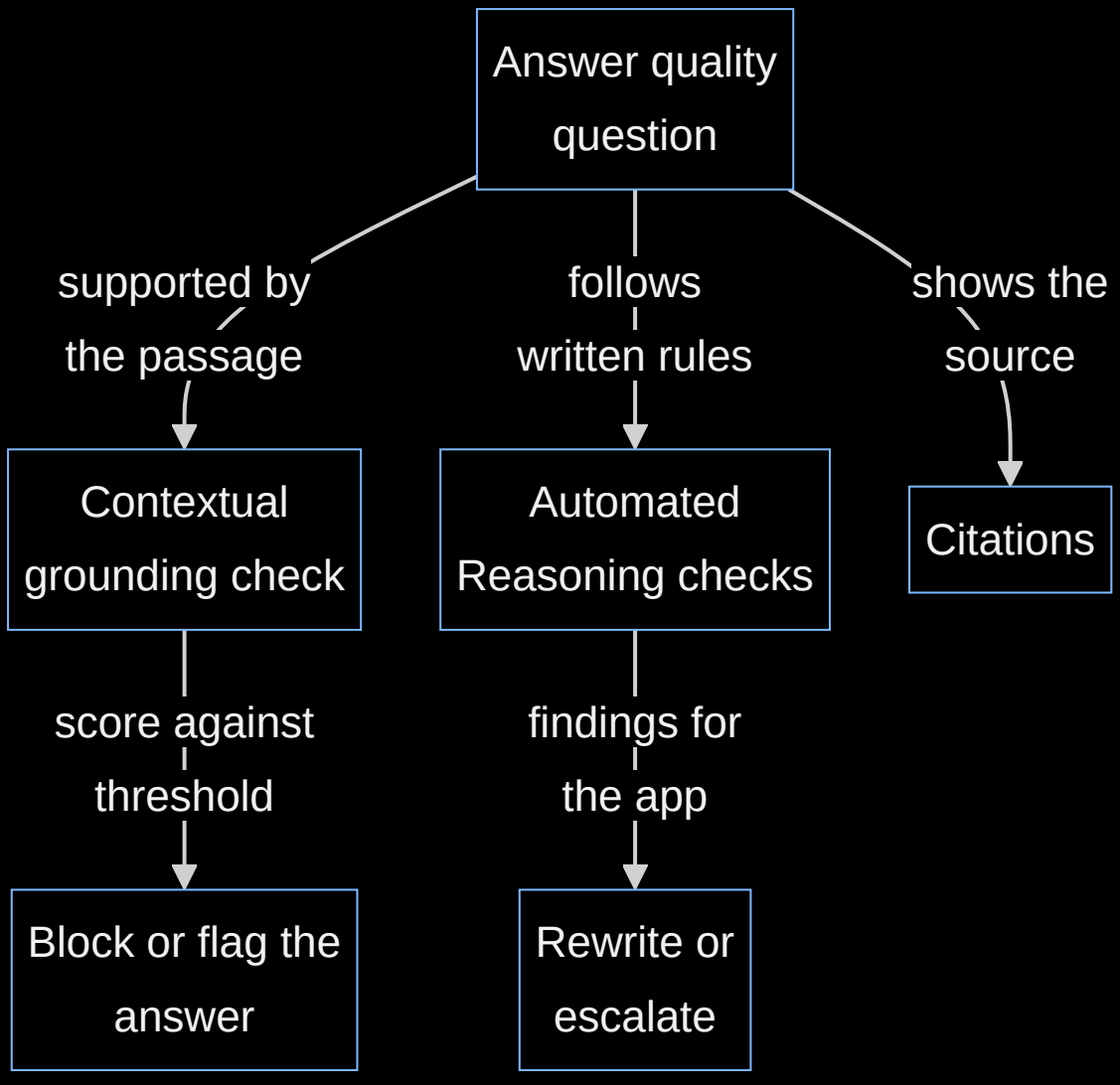


Figure: Pick the layer that matches the question in the stem.

- Purpose. Grounding: supported and relevant. Automated Reasoning: valid under rules. Citations: source links.

- Input. Grounding: source, query, and response. Automated Reasoning: policy document and response. Citations: retrieved passages.
- Output or effect. Grounding: scores and a block or flag. Automated Reasoning: findings only. Citations: references.

Exam tip: The word "rule" or "eligibility policy" points to Automated Reasoning. The word "supported by the passage" points to grounding.

Rule: Use grounding for passage support, Automated Reasoning for formal rules, and citations for traceability.

The one question to ask yourself: Is the claim wrong against a passage, or against a rule?

Trap: Treating a citation as proof of correctness. A citation can point to a passage that does not support the claim.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html.

Clarify versus model evaluation versus guardrails for bias

Bias work splits into three jobs. Clarify measures bias in data or predictions across groups. Model evaluation measures quality across a test set. Guardrails block or mask content in one live request. A bias question about a dataset does not need a guardrail, and a live harmful answer does not need a dataset metric.

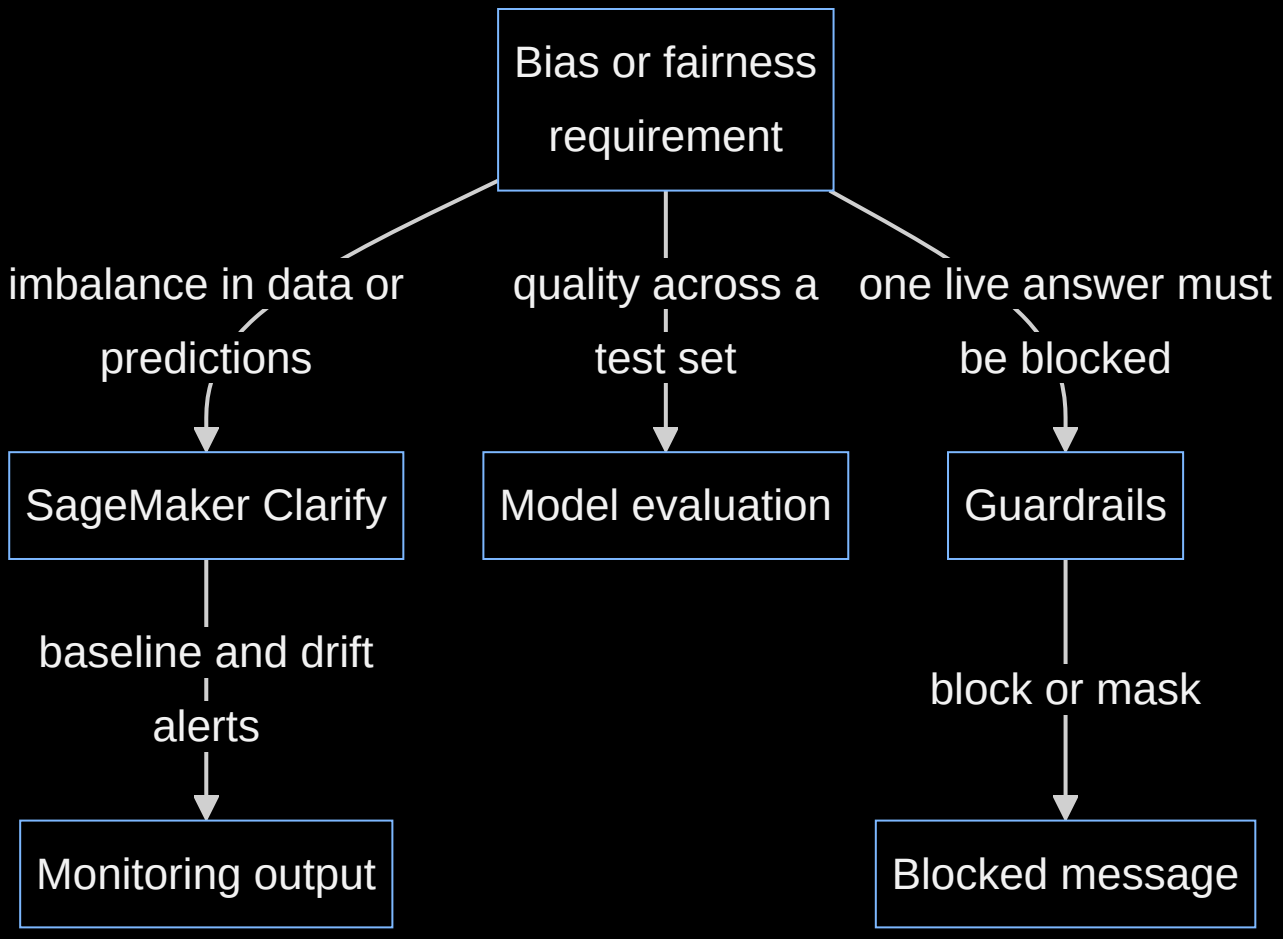


Figure: Scope decides the tool. Dataset scope, test set scope, or single request scope.

- Purpose. Clarify: measure bias. Model evaluation: measure quality. Guardrails: enforce on one request.
- Input. Clarify: a dataset or predictions. Model evaluation: prompts and responses in a test set. Guardrails: one prompt or response.
- Output or effect. Clarify: metrics and alerts. Model evaluation: scores. Guardrails: block, mask, or report.

Exam tip: Check the account status in the stem. Clarify is closed to new customers, so a new-account design should not depend on it.

Rule: Use Clarify for group bias in data or predictions, and use guardrails for a live harmful output.

The one question to ask yourself: Is the bias question about a population, or about one message?

Trap: A guardrail with a denied topic for a demographic group is not a fairness metric. It blocks text, but it does not measure disparity.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-detect-data-bias.html, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-components.html. Sundog slide pages 327 and 433.

Automated blocking versus Amazon A2I review

A guardrail decides at once. A2I sends the case to a person. Both can sit in one design. The guardrail handles the clear cases, and A2I handles the borderline cases that need judgment.

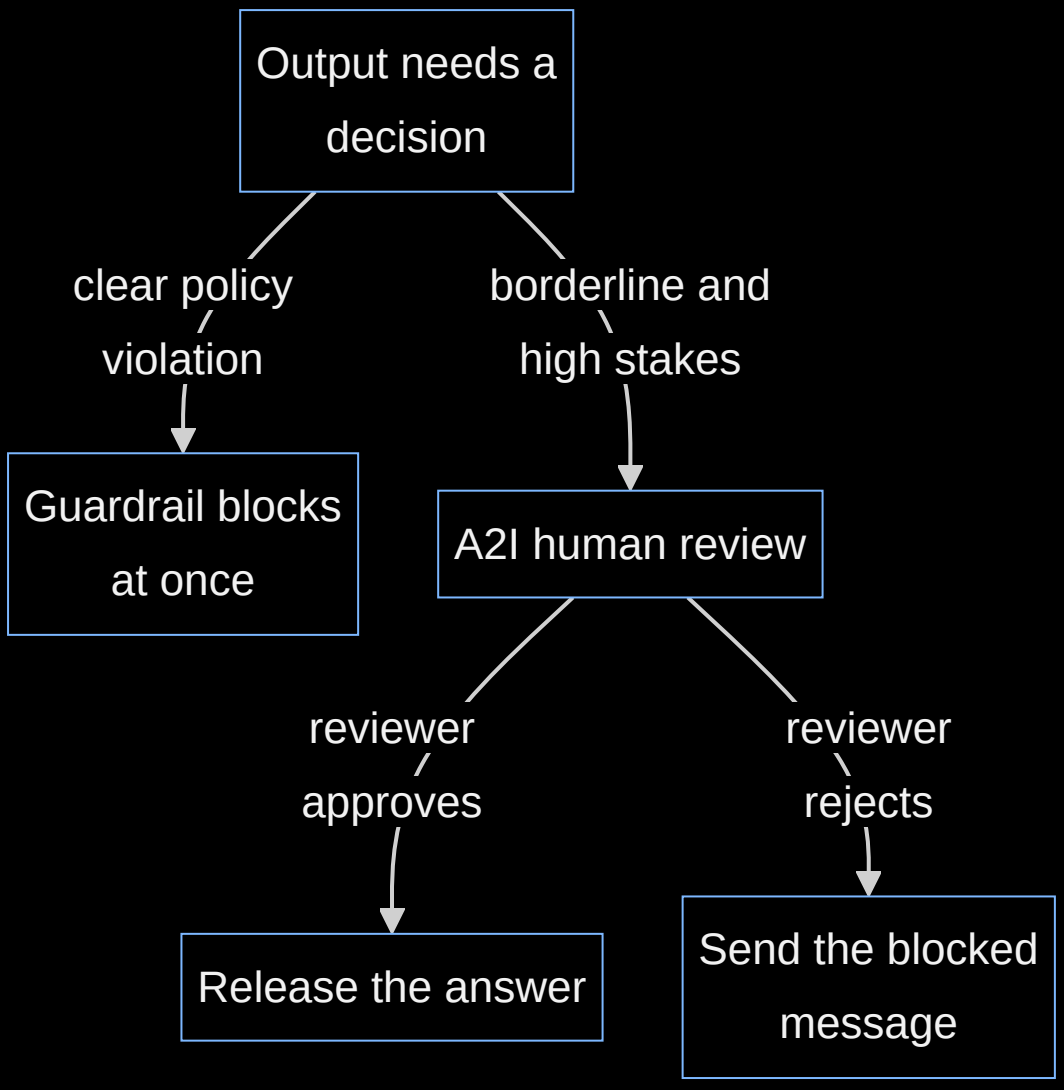


Figure: Automatic for clear cases, human for judgment cases.

- Purpose. Guardrail: enforce policy at once. A2I: add human judgment.
- Input. Guardrail: text in a request. A2I: a model result and its context.
- Output or effect. Guardrail: an immediate action. A2I: a reviewed decision.

Exam tip: Words such as "reviewer," "escalate," or "approval" point to A2I. Words such as "immediately" or "before the response is shown" point to a guardrail.

Rule: Use a guardrail for the live decision, and use A2I for the cases that a person must judge.

The one question to ask yourself: Can the decision wait for a person?

Trap: Placing A2I as the only control in a live chat path. The reviewer is too slow for that path.

Source: Sundog slide page 433. A2I documentation: not confirmed in this review.

Guardrails versus AWS Control Tower guardrails

The word "guardrail" is used for two different services. Bedrock Guardrails filters model traffic. Control Tower guardrails govern AWS accounts. Control Tower uses preventive controls built on service control policies and detective controls built on AWS Config.

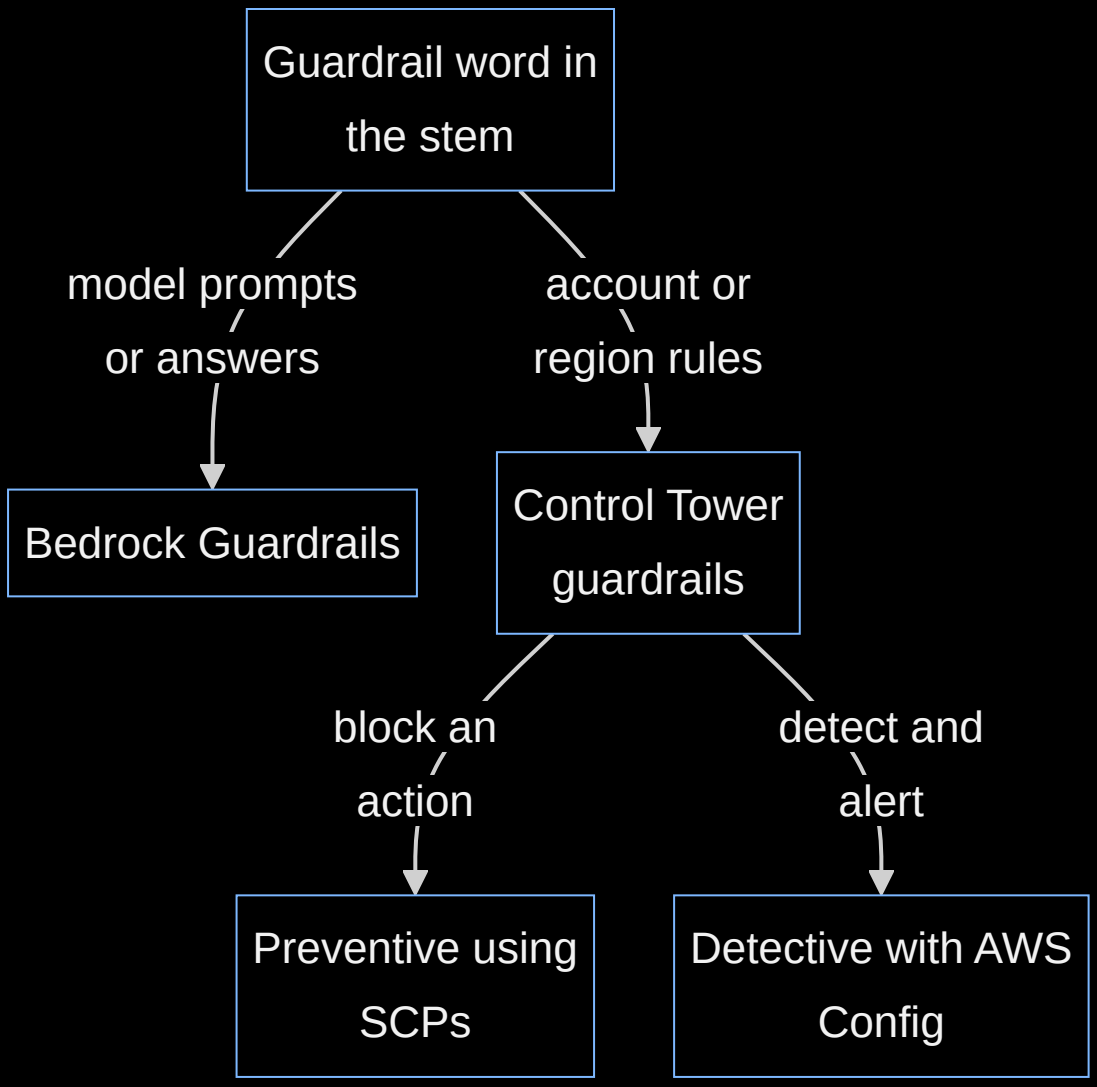


Figure: Read the object of the word guardrail before you choose.

- Purpose. Bedrock Guardrails: safety on model traffic. Control Tower guardrails: governance of accounts.
- Input. Bedrock Guardrails: prompts and responses. Control Tower: account actions and resource configuration.
- Output or effect. Bedrock Guardrails: block or mask. Control Tower: deny an action or report non-compliance.

Exam tip: If the stem names prompts, responses, tokens, or a model, the guardrail is Bedrock. If it names accounts, OUs, or regions, the guardrail is Control Tower.

Rule: Match the word guardrail to the object it protects.

Trap: Choosing a Control Tower guardrail to stop harmful model output. It does not read model text.

Source: Sundog slide pages 490 and 491 describe Control Tower guardrails with preventive and detective types. Sundog slide page 54 describes Bedrock Guardrails.

Guardrail masking versus Comprehend redaction

Both mask PII. A guardrail masks text as part of a model policy, and it does so on prompt or response. Comprehend redaction is a separate step that your design calls before the model. The difference matters when the copy is stored, because a guardrail mask does not change what the invocation log keeps.

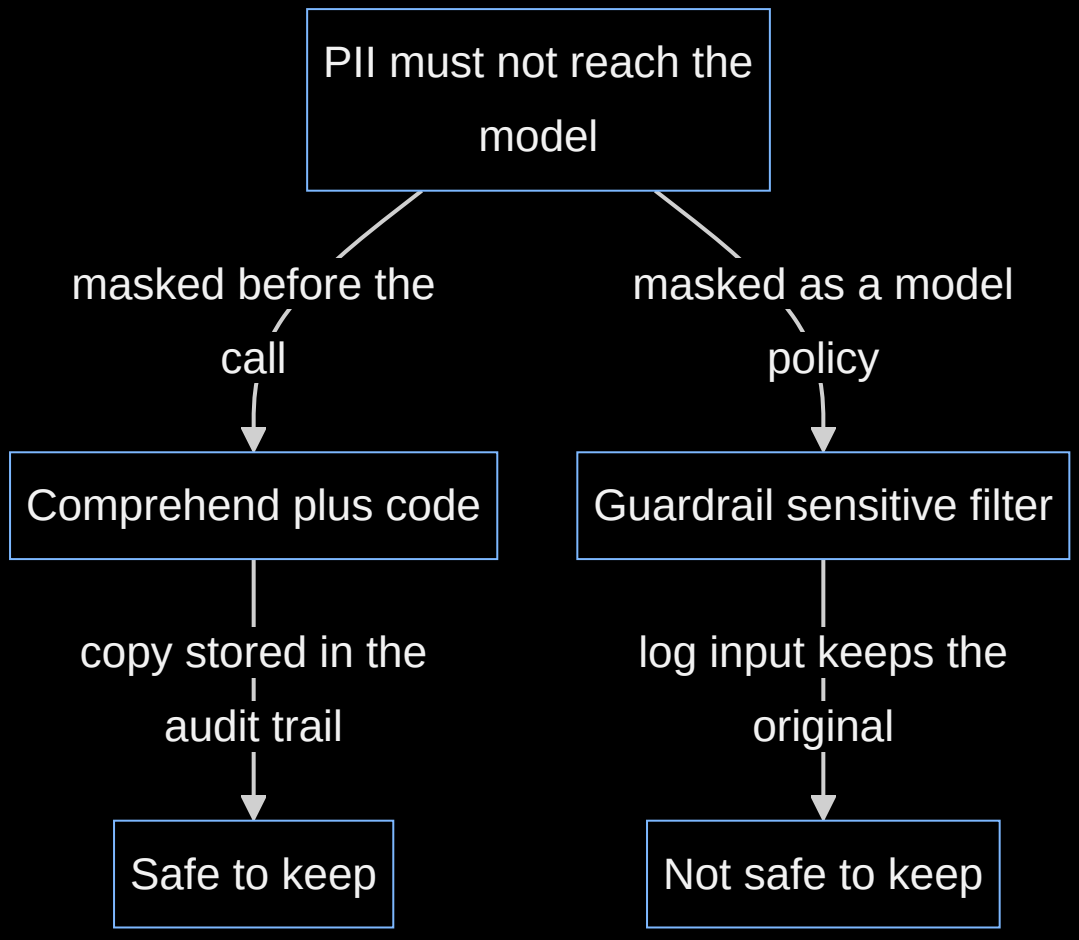


Figure: Masking in the guardrail protects the answer, not the stored input.

- Purpose. Guardrail: policy on prompt or answer. Comprehend: locate or redact before the call.
- Input. Guardrail: prompt or response text. Comprehend: text your code sends.

- Output or effect. Guardrail: masked answer, with the original kept in logs. Comprehend: redacted input before the model sees it.

Exam tip: If the stem asks what the audit log stores, the answer depends on whether the input was redacted before the call.

Rule: For clean logs, redact before the model call and use the guardrail as the second layer.

The one question to ask yourself: Does the stored record need to be clean?

Trap: Relying on guardrail masking alone when the stem requires clean invocation logs.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html, https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html.

Read a long question: three worked scenarios

Each scenario below is written for this module. The steps are the same for any long stem. Find the deciding constraint, underline the words that set it, reject each option with one reason, and then choose.

Scenario one: live chat with personal data

A support team sends each live chat message to a model on Bedrock. Customer names and phone numbers must not reach the model. Each chat message is under 10 KB.

Constraint words: live chat, must not reach the model, names and phone numbers, under 10 KB.

- Macie: rejected. It scans S3 at rest, and the message is in a live request.
- Asynchronous redaction job: rejected. It is batch, and the chat path cannot wait.
- Prompt instruction to ignore personal data: rejected. An instruction is not enforcement.

- Real-time DetectPiiEntities in Lambda, masking offsets, then the model call: chosen. It runs in the live path and works under the size limit.

Add a guardrail with a sensitive information filter on the model call. That covers output masking as a second layer.

Rule: Live text and personal data point to real-time Comprehend in code, with a guardrail as the second layer.

Scenario two: a notice assembled by code

A Lambda function builds a fixed notice from a rules engine. The notice must pass the same safety policy as the chatbot, and no model call is allowed for the notice. The team wants the least custom code.

Constraint words: assembled by code, same safety policy, no model call, least custom code.

- Second guardrail with different settings: rejected. Two policies drift apart over time.
- Model call to rewrite the notice: rejected. It breaks the no-model rule and adds cost.
- Custom regex and word lists in Lambda: rejected. It is more custom code than the managed option.
- ApplyGuardrail with the same guardrail and version: chosen. It evaluates the text without a model call.

Rule: Non-model text with an existing policy points to ApplyGuardrail on the same guardrail version.

Scenario three: an answer from policy pages

A benefits assistant answers from policy pages retrieved by RAG. The answer must be supported by the retrieved text, and a reviewer must see where each claim came from. The team does not need a formal rule engine.

Constraint words: supported by the retrieved text, reviewer must see where, no formal rule engine.

- Retrieval score as proof: rejected. A relevance score is not a truth score.
- Automated Reasoning: rejected for this stem. The team does not have a formal rule set, and the check is detect mode only.
- Word filter for wrong facts: rejected. Word filters match exact terms, not facts.
- Contextual grounding check with the passages as source, plus citations: chosen. It scores support, and citations show the source.

Rule: Support against passages points to the grounding check. Citations are for traceability only.

Distractor patterns seen in the tests

Wrong options follow a small set of patterns. Learn the pattern, and you can reject an option even when you do not know every service detail.

- Trap: The right service in the wrong place. Macie scans storage, so it does not protect a live message. Each option sounds correct, but the data location is wrong.
- Trap: The right service at the wrong time. A batch job cannot sit in a live path that must answer in seconds.
- Trap: A prompt as a control. A system prompt that says "refuse" is advice to the model. It is not a check the application can rely on.
- Trap: A location returned as a redaction. A real-time PII call returns offsets. The original text stays until code changes it.
- Trap: A score treated as truth. Retrieval scores and toxicity scores measure something other than correctness.
- Trap: A filter named for the wrong risk. A word filter for PII, or a denied topic for a list of words, uses the wrong policy type.
- Trap: A model call used as a moderator. Calling a model to judge text that a managed policy could judge adds cost and a new failure mode.
- Trap: Manual review as an automatic block. A2I routes to a person. It does not stop a response at once.

- Trap: A limit that is not real. A threshold of 1 for grounding, or a strength that is not in the list, is invalid.
- Trap: A retired or closed feature. Clarify and prompt safety classification are closed to new customers, so a design that depends on them fails the account condition.
- Trap: Guardrails assumed to cover tool results. The docs say content filters and sensitive filters do not evaluate tool results, tool definitions, or tool call arguments.

Drill

Cover the answer line, read the prompt, decide, then check.

1. Chat input must block insults before the model call. Which control?

Rule: Content filters with the Insults category, input strength set, on the guardrail attached to the call.

2. A monthly batch of files in S3 must produce redacted copies. Which service and mode?

Rule: Comprehend asynchronous redaction job. Real-time calls return offsets only.

3. Find buckets of files that hold personal data in S3. Which service?

Rule: Amazon Macie, using sensitive data discovery on S3.

4. A model must not discuss one named subject. Which policy?

Rule: Denied topics, described in natural language.

5. A list of banned product code words must be blocked exactly. Which policy?

Rule: Word filters with custom words.

6. Customer account numbers have a fixed pattern. Which guardrail feature?

Rule: Sensitive information filter with a custom regex, action block or mask.

7. A generated answer must be supported by the retrieved passages, with a score. Which check?

Rule: Contextual grounding check with the passage as the source, the question as the query, and the answer as the content.

8. A written eligibility policy with conditions must govern each answer. Which check?

Rule: Automated Reasoning checks, which return findings in detect mode.

9. Text from a rules engine must be checked with no model call. Which API?

Rule: ApplyGuardrail with the guardrail identifier, version, and source.

10. A tool result returned by an external API may contain injected instructions. Which control is missing?

Rule: Application validation of the tool result in Lambda, since the prompt attack filter does not evaluate tool results.

11. Auditors need the prompt and response record for each Region. Which setting?

Rule: Model invocation logging per Region, with the same-account destination. It is off by default.

12. The stem says a person must approve borderline outputs before release. Which service?

Rule: Amazon A2I human review loop, not a guardrail block.

Sources

- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-components.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters-overview.html

- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-prompt-attack.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-use-independent-api.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html
- https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html
- https://docs.aws.amazon.com/comprehend/latest/dg/trust-safety.html
- https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html
- https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-detect-data-bias.html
- Sundog course slide extract, module m2 (corpus file m2-safety-pii-sundog.txt), slide pages 21, 54, 55, 57, 69, 118, 162, 264, 281, 327, 328, 330, 373, 419, 433, 446, 490, 491, 499.